

基于 MPI 的分布式 ANN 检索并行化

结合 SIMD、OpenMP 混合并行与图索引组合的实验分析

姓名	匡航远	学号	2412235
专业	计算机科学与技术	日期	2026 年 5 月 30 日
GitHub	https://github.com/xzxxntxdy/Parallel-programming-NKU		

摘要

本文围绕 DEEP100K 向量检索任务实现并分析了 ANN 查询阶段的 MPI 并行化。实现采用数据分片方案: rank 0 读取 base/query/ground truth, base vectors 通过 `MPI_Scatterv` 按连续区间分发到各 MPI 进程, query 通过 `MPI_Bcast` 广播; 每个 rank 在本地分片上构建 IVF 或 HNSW 索引, 产生局部 top-100 候选, 最终由 rank 0 合并为全局 top-100 并计算 **recall@100**。在此基础上, 实验进一步实现了 Flat、IVF、HNSW、multi-HNSW 与 IVF-HNSW, 比较了 blocking/nonblocking/point-to-point/one-sided 通信、`MPI_THREAD_FUNNELED`/`MPI_THREAD_MULTIPLE`、MPI+OpenMP 混合并行以及 scalar/SIMD 距离核。

本地 x86 正式实验共记录 172 组结果, 统一以 $\text{recall@100} \geq 0.95$ 下 **latency** 最小为选择准则。全量 DEEP100K 上, 单进程 OpenMP-HNSW 达到 0.0312 ms/query, recall@100 为 0.9545; 限定 MPI 并行 rank ≥ 2 时, 2 ranks + 4 OpenMP threads 的 HNSW 达到 0.0330 ms/query, recall@100 为 0.9816。IVF 在 8 ranks、nprobe=32 时达到 0.0573 ms/query, recall@100 为 0.9654, 是倒排索引路径的最优点。实验表明, MPI 数据分片能明显降低本地扫描量, 但当局部搜索已很快时, 结果通信和全局 merge 会成为主要瓶颈; 图索引路径的扩展性受每个 rank 内部 HNSW 搜索代价和 top-k 合并开销共同约束。

目录

1 问题定义与评价指标	3
1.1 ANN 检索目标	3
1.2 并行化目标	3
2 MPI 数据划分与全局 top-k 合并	3
2.1 数据分发策略	3
2.2 局部候选维护与全局合并	4
3 本地索引与并行实现	4
3.1 Flat 与 SIMD 距离核	4
3.2 IVF: 本地倒排索引与 nprobe 消融	4
3.3 HNSW 与 MPI 图索引组合	5
3.4 MPI + OpenMP 混合并行	5
3.5 MPI 通信模式	5

4	实验设置	5
4.1	平台与编译	5
4.2	实验矩阵	6
5	总体结果与 recall-latency 权衡	7
6	IVF 消融与 MPI 扩展性	9
6.1	nprobe 对 recall 与 latency 的影响	9
6.2	问题规模与进程数	10
7	MPI 通信方法与线程支持	11
7.1	通信方式对比	11
7.2	MPI_THREAD_MULTIPLE	11
8	OpenMP 混合并行与 SIMD 贡献	12
9	图索引组合分析	14
10	性能分析与进一步优化	14
10.1	负载均衡	14
10.2	cache locality	14
10.3	通信和 merge	15
10.4	算法选择	15
11	结论	15
A	生成式 AI 辅助学习记录	16

1 问题定义与评价指标

1.1 ANN 检索目标

给定向量集合 $X = \{x_1, \dots, x_N\} \subset \mathbb{R}^d$ 和查询向量 q , 精确 k 近邻要求返回距离最小的 k 个向量。DEEP100K 使用内积相似度, 本文沿用已有 SIMD 阶段的距离定义:

$$\delta(q, x) = 1 - \sum_{i=1}^d q_i x_i. \quad (1)$$

距离越小表示相似度越高。ANN 的目标不是穷尽所有精确候选, 而是在可接受召回下减少查询访问的数据量和单 query 延迟。

本文固定 $k = 100$, 以 recall@100 衡量质量:

$$\text{recall@100}(q) = \frac{|R_{100}(q) \cap G_{100}(q)|}{100}, \quad (2)$$

其中 R_{100} 是算法返回结果, G_{100} 是 ground truth。所有正式最优点只在全量 100k base 上比较; 25k/50k 规模实验用于观察计算量、通信量和进程数变化趋势。

1.2 并行化目标

MPI 的核心目标是把 base data 分布到多个进程, 使每个进程只搜索自己的本地分片。该策略不会重复完整搜索任务, 单 query 的本地计算量从 $O(Nd)$ 降为近似 $O(Nd/P)$ 。代价是每个 query 需要收集 P 个局部 top-100, 然后做全局 merge。对 IVF 而言, 单 rank 的主要计算复杂度为

$$O(nlist \cdot d + nprobe \cdot \bar{L} \cdot d), \quad (3)$$

其中 $\bar{L}$ 是本地倒排链平均长度。通信和 merge 复杂度为

$$O(Pkd) \text{ 的候选距离/编号传输近似为 } O(Pk), \quad O(Pk \log k) \quad (4)$$

的堆式合并。由于 $k = 100$ 固定, 增大 P 后本地搜索下降, 但通信和 merge 会逐步显现。

2 MPI 数据划分与全局 top-k 合并

2.1 数据分发策略

实现采用 base-sharding 而不是 query-sharding。rank 0 负责读取数据集元信息、base、query 和 ground truth, 然后广播矩阵规模。base data 按连续区间切分, 每个 rank 获得 $[l_r, r_r)$ 区间:

$$|X_r| \in \left\{ \left\lfloor \frac{N}{P} \right\rfloor, \left\lceil \frac{N}{P} \right\rceil \right\}. \quad (5)$$

这种划分的优势是分片大小天然均衡, 全局 id 可以由 `global_begin + local_id` 得到, 不需要额外 id 映射表。query 被广播到所有 rank, 保证每个 rank 对相同 query 产生局部候选。

在代码中, 该流程由 `distribute_data` 完成。关键 MPI 操作包括:

```
1 MPI_Bcast(meta, 4, MPI_UNSIGNED_LONG_LONG, 0, MPI_COMM_WORLD);
2 MPI_Bcast(query.data(), query.size(), MPI_FLOAT, 0, MPI_COMM_WORLD);
3 MPI_Bcast(gt.data(), gt.size(), MPI_INT, 0, MPI_COMM_WORLD);
4 MPI_Scatterv(root_base, counts, displs, MPI_FLOAT,
5             local_base.data(), counts[rank], MPI_FLOAT, 0, MPI_COMM_WORLD);
```

query-sharding 的通信量更低，但每个 query 只会在单个 rank 上被搜索，除非每个 rank 都保存完整索引，否则无法得到全局 top-k。本文选择 base-sharding，是为了保证召回定义和单机版本一致，同时明确暴露通信、merge 与负载均衡问题。

2.2 局部候选维护与全局合并

每个 rank 对每个 query 维护固定大小的 top-100 最大堆。堆顶保存当前候选中距离最大的元素，当新候选距离更小时替换堆顶。局部结果被转成两个固定长度数组：`local_dist[nq*k]` 和 `local_ids[nq*k]`。固定长度布局使 gather 通信不需要额外交换每个 query 的候选数，便于比较不同 MPI 通信方式。

Algorithm 1 MPI base-sharding ANN 查询

- 1: rank 0 loads base/query/ground truth.
 - 2: Broadcast query and metadata; scatter base shards to all ranks.
 - 3: Each rank builds local IVF/HNSW index on its shard.
 - 4: **for** query q **do**
 - 5: each rank searches local index and keeps local top-100 heap.
 - 6: **end for**
 - 7: Gather fixed local candidates to rank 0.
 - 8: rank 0 merges $P \times 100$ candidates per query into global top-100.
 - 9: rank 0 computes recall@100 using ground truth.
-

全局 merge 仍使用堆维护 top-100。该合并逻辑只依赖候选距离和全局 id，因此 Flat、IVF、HNSW、multi-HNSW、IVF-HNSW 共享同一套结果汇总路径。CSV 同时记录 `search_ms`、`comm_ms`、`merge_ms`，后文的 profiling 分析直接来自这些阶段计时。

3 本地索引与并行实现

3.1 Flat 与 SIMD 距离核

Flat 路径用于串行和 MPI 精确扫描基线。每个 rank 顺序扫描自己的本地 base 分片，距离计算调用 `ann::ip_distance`。为了分析 SIMD 对 MPI 版本的贡献，程序增加了 `--kernel` 参数：

- `scalar`: 禁止编译器自动向量化的标量内积；
- `auto`: 交给编译器自动向量化；
- `simd`: x86 上选择 AVX，ARM 上选择 NEON unroll4，其他平台退化到自动向量化。

x86 Windows 构建使用 `/arch:AVX2`，Linux/ARM 使用 `-march=native`。这样同一份 MPI 代码可以在 x86 与 ARM 上走对应 SIMD 路径。

3.2 IVF: 本地倒排索引与 nprobe 消融

IVF 在每个 rank 的本地分片上独立构建倒排索引。构建阶段使用本地 k-means 初始化和迭代，搜索阶段先计算 query 到本地所有 centroid 的距离，选择最近的 `nprobe` 个倒排链，再扫描链中向量。该设计没有跨 rank 的全局 centroid 同步，构建成本低、通信路径简单，但不同 rank 的局部簇边界不完全一致。因此 IVF 的有效搜索宽度不仅由 `nprobe` 决定，也受数据分片影响。

搜索函数 `search_local_ivf_batch` 在 query 维度上并行。每个 query 内部的倒排链访问保持顺序，减少锁和跨线程堆合并开销。平均访问候选数被记录为 `local_work`，再通过 `MPI_Reduce` 得到 work imbalance。全量实验中 IVF 的 work imbalance 约为 1.03，说明连续切分下各 rank 的访问量比较接近，主要瓶颈不在数据量倾斜，而在通信与 merge。

3.3 HNSW 与 MPI 图索引组合

图索引部分实现三条路径。第一条是 HNSW：每个 rank 在本地分片上构建一个 HNSW，搜索同一 query 后由 rank 0 合并。第二条是 multi-HNSW：数据集被 MPI 分片后形成多个 HNSW，概念上等价于每个 rank 负责一个子图。第三条是 IVF-HNSW：每个 rank 先构建本地 IVF，再在每个倒排簇内建立 HNSW，搜索时先筛选 `nprobe` 个簇，再在簇内图上搜索。

HNSW 的优势是访问点数量远少于 IVF 扫描，在 `recall@100` 阈值附近可以取得最低延迟。其代价是构建时间明显高于 Flat/IVF，且 query 内部搜索带有图遍历依赖，难以像矩阵扫描那样细粒度拆分。本文因此采用 query-level OpenMP 并行：同一 rank 内多个 query 并发进入本地 HNSW，避免在单 query 图遍历中引入高同步开销。

3.4 MPI + OpenMP 混合并行

混合并行由 `parallel_for_queries` 统一封装。当 `--thread-model openmp` 时，使用：

```
1 #pragma omp parallel for num_threads(threads) schedule(dynamic)
2 for (long long qi = 0; qi < (long long)nq; ++qi) {
3     search_one_query(qi);
4 }
```

当 `--thread-model stdthread` 时，使用 C++ worker 线程按 query id 交错分配任务。OpenMP 的 dynamic schedule 更适合 HNSW，因为不同 query 的图遍历步数存在波动；IVF 的负载更稳定，线程调度开销相对更显著。

3.5 MPI 通信模式

局部结果回收实现了四种通信模式：

- **blocking**: 使用 `MPI_Gather` 收集距离数组和 id 数组；
- **nonblocking**: 使用两次 `MPI_Igather` 并 `MPI_Waitall`；
- **point-to-point**: 非 root rank 用 `MPI_Send`，root 用 `MPI_Recv`；
- **one-sided**: root 创建 `MPI_Win`，其他 rank 使用 `MPI_Put` 写入窗口。

此外，程序可通过 `--mpi-thread-multiple` 请求 `MPI_THREAD_MULTIPLE`，用于比较 MPI 运行时线程支持带来的开销。实际 MS-MPI 返回的 `provided` 等级为 3，即支持 `MPI_THREAD_MULTIPLE`。

4 实验设置

4.1 平台与编译

本地正式实验在 Windows x86 环境完成。处理器为 Intel64 Family 6 Model 183，逻辑处理器数 32；MPI 运行时为 MS-MPI 10.1.12498.16；编译器为 MSVC 19.43，编译选项包含

/O2 /arch:AVX2 /openmp /std:c++17。MPI SDK 与运行时通过本地 Conda 环境提供，路径为 .conda-envs/mpi/Library。

正式命令为：

```
1 cd ann
2 powershell -NoProfile -ExecutionPolicy Bypass -File .\run_mpi_x86.ps1 -Clean
3 python .\plot_mpi_report.py
```

ARM/Kunpeng 侧保留 quick/full 命令，编译使用 -O3 -fopenmp -march=native，以便在 AArch64 上自动启用 NEON：

```
1 cd /home/$USER/ann
2 ANN_DATA=/anndata bash run_mpi_arm_kunpeng.sh --quick
3 ANN_DATA=/anndata CSV=files/results/mpi_results_arm_kunpeng_full.csv \
4     bash run_mpi_arm_kunpeng.sh --full
```

PBS 集群提交入口为：

```
1 NP=16 ANN_DATA=/anndata CSV=files/results/mpi_results_arm_kunpeng_cluster.csv \
2     qsub qsub_mpi.sh
```

4.2 实验矩阵

本地正式 CSV 为 files/results/mpi_results_x86_windows.csv，共 172 行。主要覆盖如下：

- base 规模：25k、50k、100k；query 数：1000；
- MPI ranks：1、2、4、8；
- OpenMP threads per rank：1、2、4；
- IVF nprobe：16、32、64、128、256；
- 通信模式：blocking、nonblocking、point-to-point、one-sided；
- MPI 线程等级：FUNNELED 与 MULTIPLE；
- 距离核：scalar 与 SIMD；
- 算法族：Flat、IVF、HNSW、multi-HNSW、IVF-HNSW。

每组正式实验重复 3 次，记录最低 latency。最低值更接近稳定运行时上界，且能降低 Windows 后台扰动对短 query 的影响。

5 总体结果与 recall-latency 权衡

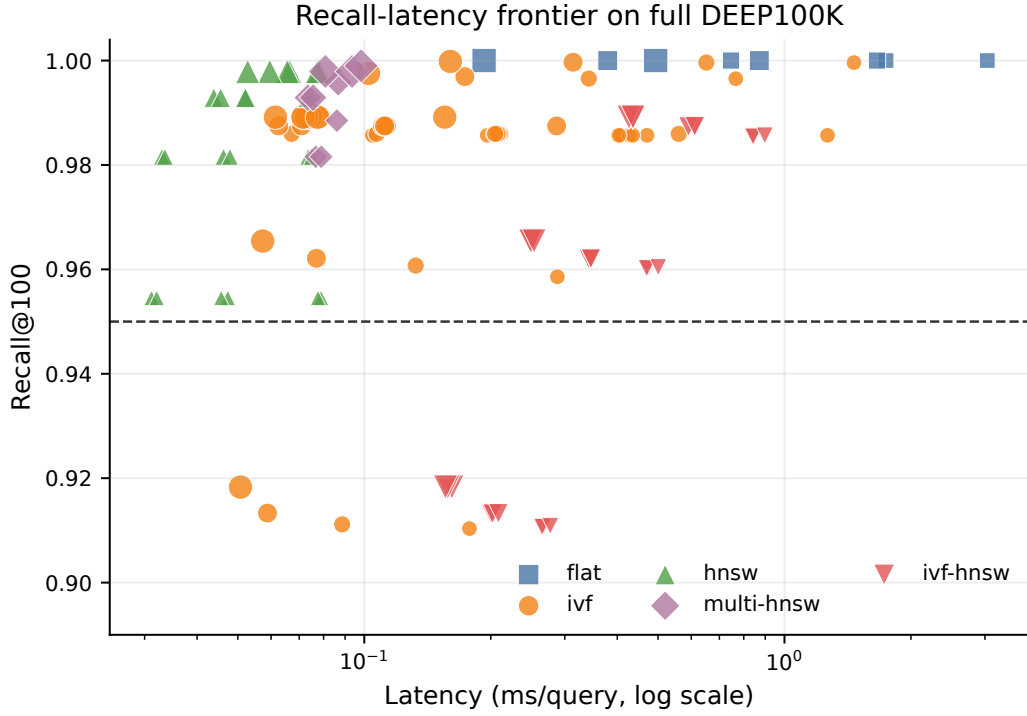


图 1: 全量 DEEP100K 上所有 100k/1000-query 实验的 recall-latency 分布。虚线为 recall@100=0.95。

图 1 显示，Flat 精确扫描稳定达到 recall@100=1.0，但延迟较高；IVF 通过调整 nprobe 在 0.91 到 1.0 之间形成连续曲线；HNSW 与 multi-HNSW 位于左上区域，是高召回低延迟的主要候选；IVF-HNSW 在当前实现中延迟明显高于 HNSW，说明簇内建图并不能抵消多簇搜索和候选合并的开销。

表 1: 各算法族在 recall@100 $\geq$ 0.95 下的最优配置

算法	ranks	threads	nprobe	ef	线程模型	通信	latency	recall
HNSW	1	4	64	64	OpenMP	nonblocking	0.0312	0.9545
IVF	8	1	32	64	stdthread	blocking	0.0573	0.9654
multi-HNSW	4	1	64	32	stdthread	nonblocking	0.0734	0.9929
Flat	8	1	512	64	stdthread	blocking	0.1928	1.0000
IVF-HNSW	8	1	32	64	stdthread	nonblocking	0.2488	0.9654

表 1 给出按统一准则筛出的最优点。若不强制使用多 rank，HNSW 的单进程 4 OpenMP threads 最快；如果要求 MPI rank $\geq$ 2，最优点是 2 ranks + 4 OpenMP threads，latency 为 0.0330 ms/query，recall@100 为 0.9816。两者差距很小，说明 HNSW 的单 rank 并发已经足够快，多 rank 主要提升 recall 和构建分片规模，而不是继续降低查询延迟。

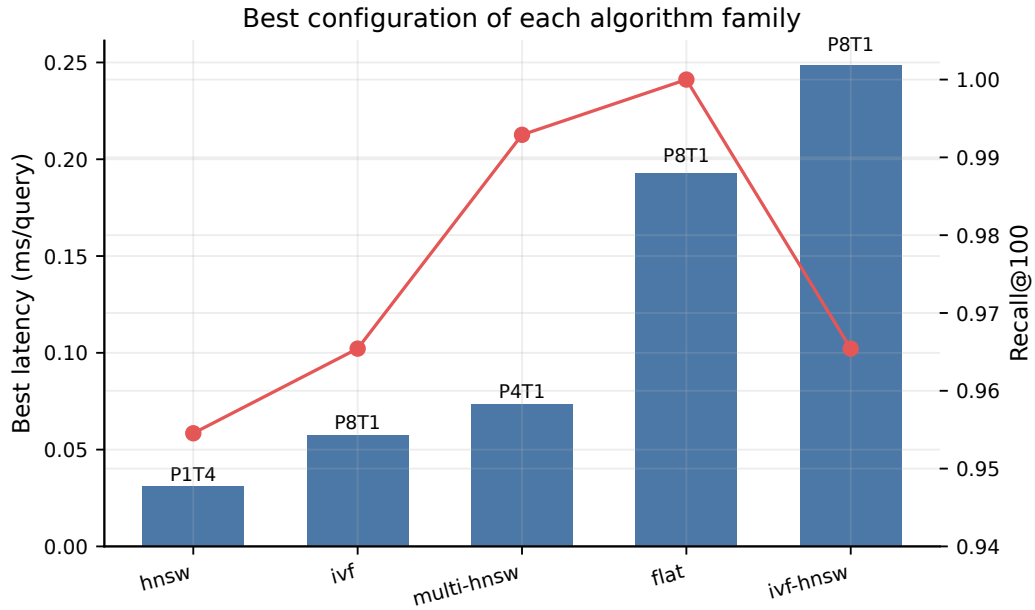


图 2: 各算法族最优配置的 latency 和 recall。柱顶的 P/T 表示 MPI ranks 与每 rank 线程数。

从图 2 可以看到，IVF 是最稳定的倒排索引方案，延迟约为 Flat 的 29.7%；HNSW 进一步将延迟降低到 Flat 的 16.2%。multi-HNSW 的 recall 更高，但每个 rank 都需要完成本地图搜索并传回 top-100，merge 成本抵消了部分收益。IVF-HNSW 的 recall 能通过 $nprobe=32$ 达到阈值，但搜索多个簇内 HNSW 的开销明显高于直接使用一个本地 HNSW。

6 IVF 消融与 MPI 扩展性

6.1 nprobe 对 recall 与 latency 的影响

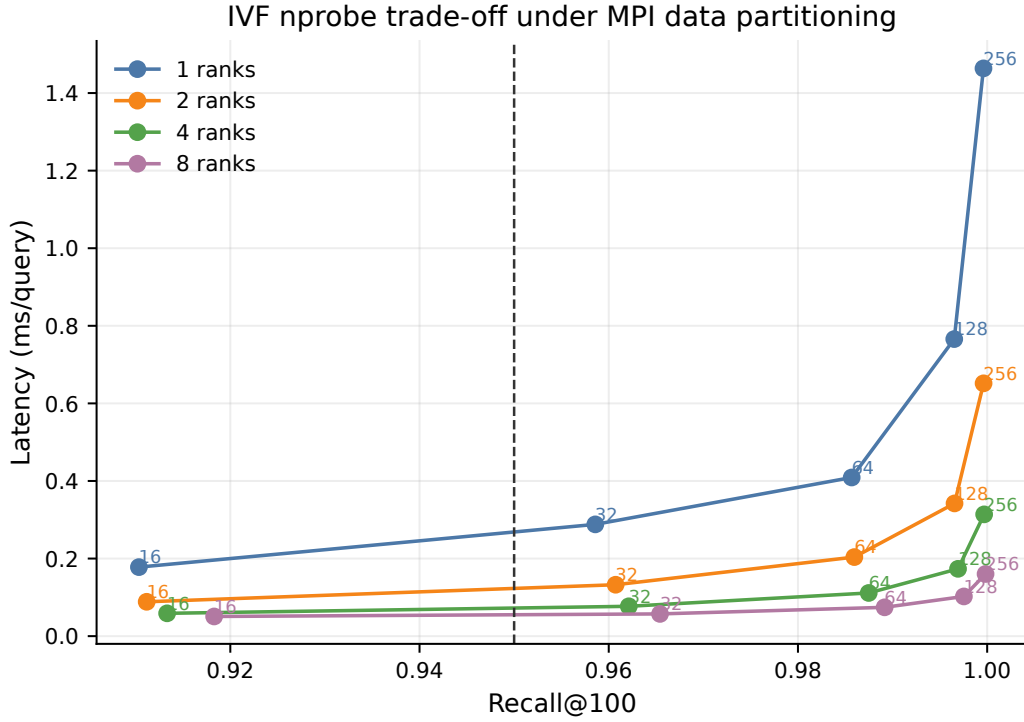


图 3: IVF 在不同 MPI ranks 下的 nprobe 消融。点旁数字为 nprobe。

IVF 的主要质量-性能旋钮是 nprobe。表 2 进一步列出各 rank 在 nprobe 消融下的结果。nprobe=16 延迟最低但 recall 低于 0.95；nprobe=32 在 8 ranks 下达到 0.0573 ms/query 和 0.9654 recall，是 IVF 路径的最优点；继续增大到 64/128/256 会提高召回，但 latency 近似线性增加。

表 2: IVF nprobe 消融: latency / recall

ranks	nprobe=16	nprobe=32	nprobe=64	nprobe=128	nprobe=256
1	0.178/0.910	0.288/0.959	0.409/0.986	0.766/0.997	1.464/1.000
2	0.089/0.911	0.133/0.961	0.204/0.986	0.342/0.997	0.652/1.000
4	0.059/0.913	0.077/0.962	0.111/0.987	0.174/0.997	0.314/1.000
8	0.051/0.918	0.057/0.965	0.074/0.989	0.102/0.998	0.160/1.000

从复杂度看，nprobe 增大时访问候选数随倒排链数增长。MPI 分片使每条本地倒排链变短，因此同一 nprobe 在更多 ranks 下延迟下降。与此同时，全局 recall 会略微上升，因为每个 rank 独立返回 top-100，rank 0 实际上从 $P \times 100$ 个候选中合并，候选池更大。但这个收益不是免费获得的：候选池越大，通信和 merge 的固定成本越明显。

6.2 问题规模与进程数

表 3: Flat-SIMD 在不同 base 规模和 MPI ranks 下的 latency

base 规模	1 rank	2 ranks	4 ranks	8 ranks
25k	0.299	0.123	0.081	0.066
50k	0.561	0.233	0.137	0.104
100k	1.745	0.746	0.380	0.193

表 3 使用 Flat-SIMD 观察纯数据扫描随规模和进程数变化的趋势。100k base 下 8 ranks 相对 1 rank 的加速比为 $9.05\times$ ，超过理想 $8\times$ 的原因是每个进程的局部分片更小，top-k 堆和数据访问更容易留在 cache 中。25k base 下扩展性较弱，8 ranks 相对 1 rank 只有 $4.52\times$ ，因为本地扫描已很短，通信和 merge 占比更高。

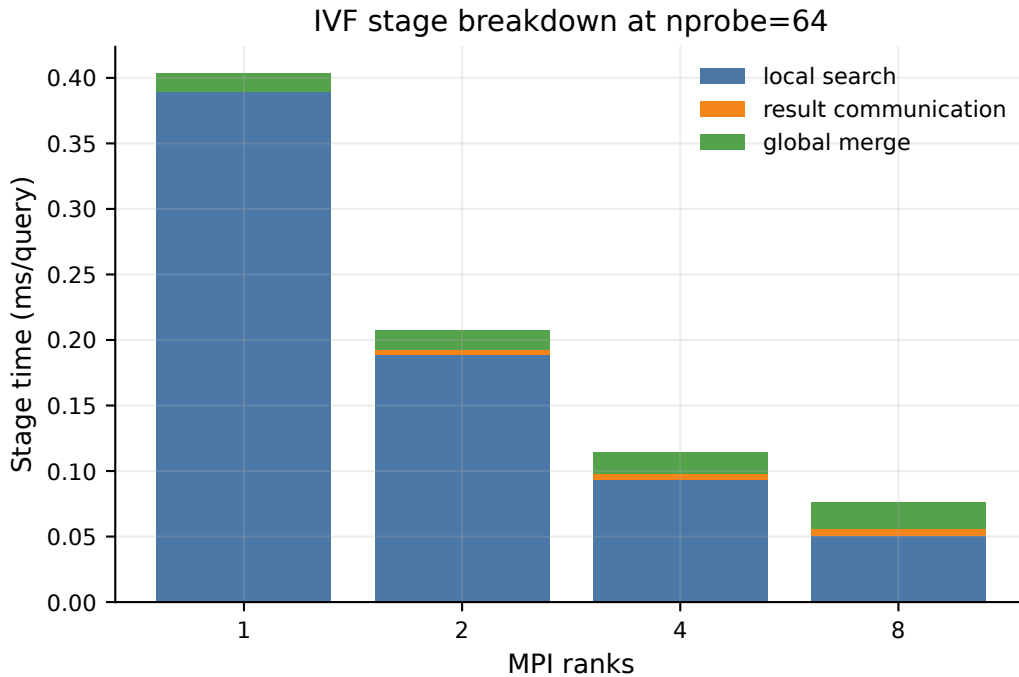


图 4: IVF nprobe=64 的阶段耗时拆分。纵轴为每 query 的 search、communication、merge 时间。

图 4 说明了 MPI 加速的瓶颈迁移：rank 从 1 增加到 8 时，本地 search 从约 0.393 ms/query 降到 0.050 ms/query；merge 从约 0.015 ms/query 增加到 0.020 ms/query；通信从约 0.0002 ms/query 增加到 0.0059 ms/query。搜索仍是主要部分，但 merge 与通信的相对占比快速上升，解释了为什么 IVF 在 8 ranks 之后预期继续扩展会变得困难。

7 MPI 通信方法与线程支持

7.1 通信方式对比

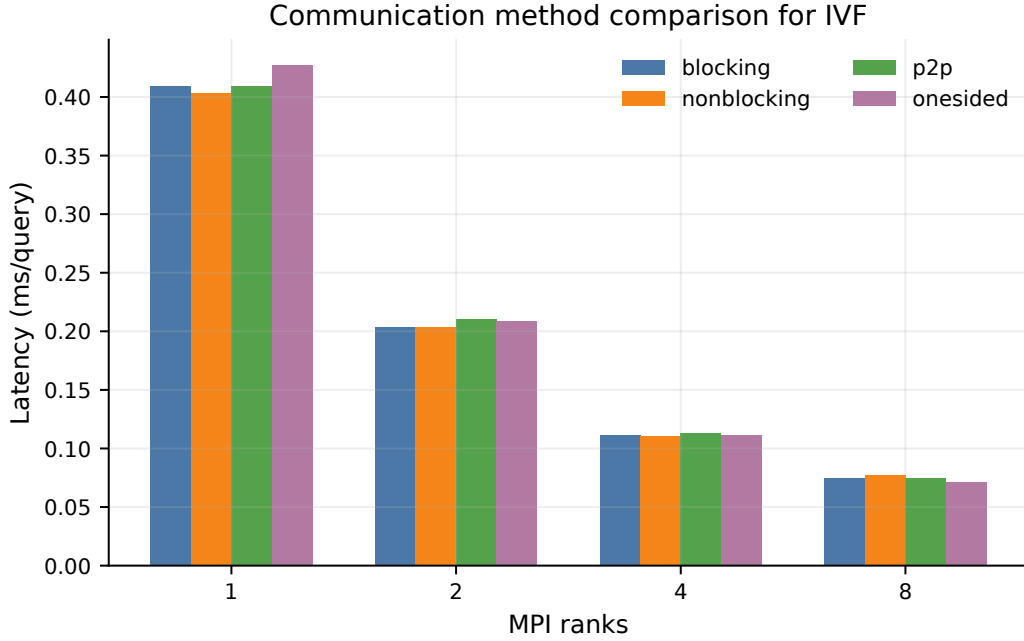


图 5: IVF nprobe=64、stdthread、SIMD 下四种结果回收通信方式的对比。

四种通信方式在单机 MS-MPI 上差距不大，因为传输对象只是 $nq \times k$ 的距离和 id 数组，通信量相对 base 数据扫描较小。表 4 给出更细的结果。1 rank 时通信方式没有真实进程间传输，one-sided 反而由于窗口创建和 fence 开销变慢。8 ranks 时 one-sided 延迟最低，为 0.0716 ms/query；p2p 和 blocking 接近；nonblocking 并没有明显优势。

表 4: IVF nprobe=64 通信方式对比：latency，括号内为总通信时间 ms

ranks	blocking	nonblocking	p2p	one-sided
1	0.409 (0.2)	0.404 (0.2)	0.409 (0.2)	0.428 (5.0)
2	0.204 (4.2)	0.204 (0.4)	0.211 (2.0)	0.209 (3.4)
4	0.111 (2.5)	0.110 (4.4)	0.113 (3.9)	0.112 (3.0)
8	0.074 (10.2)	0.077 (8.5)	0.075 (6.6)	0.072 (5.9)

nonblocking 的理论优势来自通信与计算重叠，但当前实现中本地搜索结束后才统一 gather，所以 `MPI_Igather` 只能减少调用阻塞形式带来的局部等待，不能真正隐藏搜索时间。若要进一步利用 nonblocking，需要把 query batch 分块，例如搜索第 b 个 batch 时异步传输第 $b-1$ 个 batch 的候选；这会增加结果缓存和 merge 调度复杂度，也会改变 CSV 中 stage timing 的定义。

7.2 MPI_THREAD_MULTIPLE

程序默认请求 `MPI_THREAD_FUNNELED`，即只有主线程调用 MPI。额外实验请求 `MPI_THREAD_MULTIPLE`，MS-MPI 返回 `provided=3`，说明运行时支持多个线程同时调用 MPI。结果如表 5。

表 5: MPI_THREAD_FUNNELED 与 MPI_THREAD_MULTIPLE 对比

ranks	FUNNELED latency	MULTIPLE latency	provided
1	0.4068	0.4035	3
2	0.2040	0.2049	3
4	0.1105	0.1118	3
8	0.0779	0.0774	3

差异均在短时间测量波动范围内。原因是当前设计只有主线程进入 gather/merge，工作线程只负责本地 query 计算，因此 MPI_THREAD_MULTIPLE 的能力没有被充分使用。它适合更复杂的流水线方案，例如一个线程持续搜索、另一个线程持续发起 MPI_Isend；在本实现中，FUNNELED 更简单，也避免 MPI 运行时为多线程互斥付出额外成本。

8 OpenMP 混合并行与 SIMD 贡献

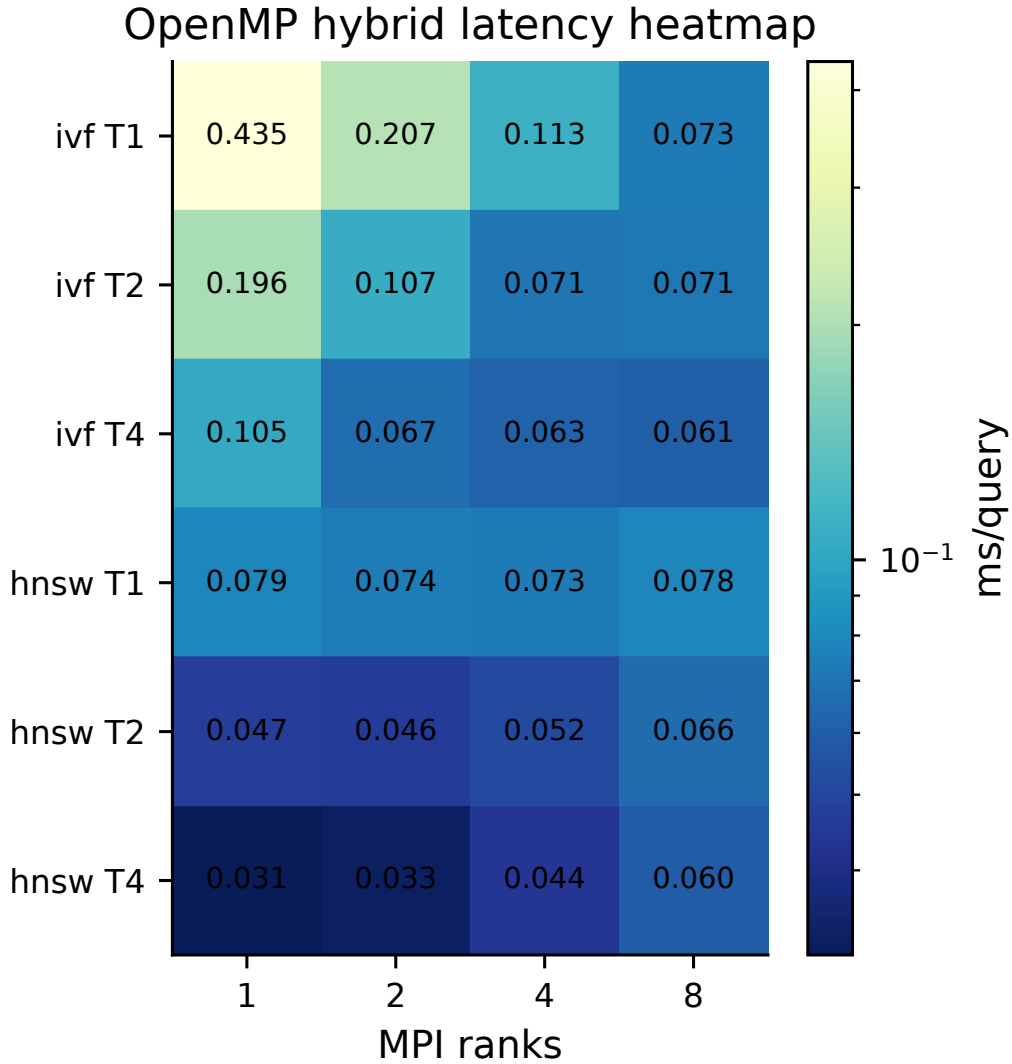


图 6: OpenMP 混合并行延迟热力图。行表示算法和每 rank 线程数，列表示 MPI ranks。

图 6 显示了进程数和线程数的 trade-off。IVF 从 T1 到 T4 持续受益，1 rank 下从 0.435 降到 0.105 ms/query；8 ranks 下 T2 与 T4 的差距很小，说明本地工作量已经被 MPI 分片切小，继续增加线程会遇到调度和 cache 竞争。HNSW 在 T4 时 1–2 ranks 最快，但 8 ranks 变慢到 0.060 ms/query，主要因为每个 rank 的本地图更小，查询本身已短，OpenMP 调度和全局 merge 成为主要开销。

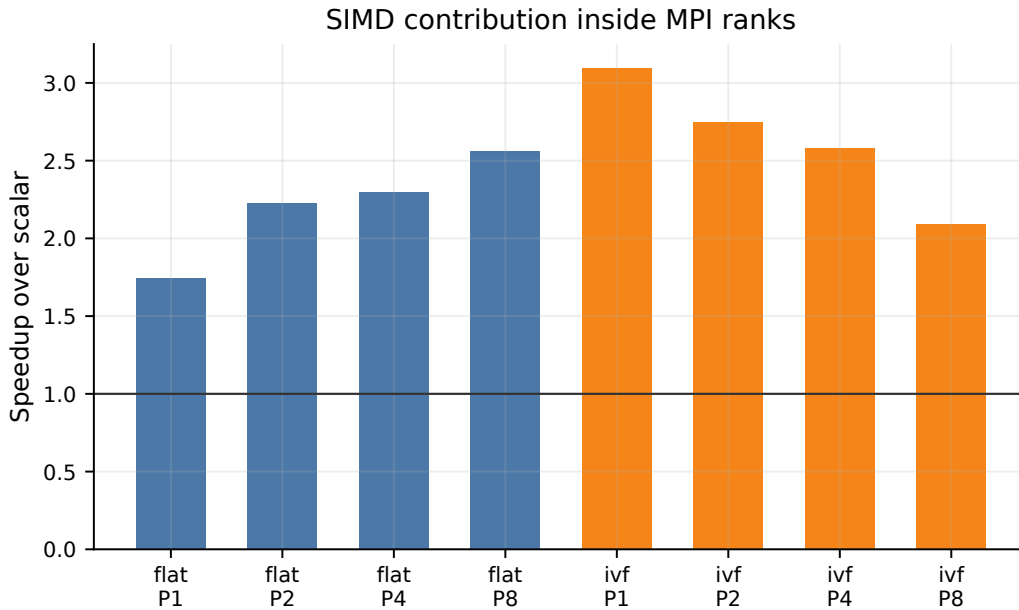


图 7: Flat 与 IVF 在不同 MPI ranks 下的 SIMD 加速比。

SIMD 对 Flat 的加速比随 ranks 增大从 $1.74\times$ 上升到 $2.56\times$ ；对 IVF 的加速比从 $3.10\times$ 下降到 $2.10\times$ 。Flat 中堆维护和 merge 占比较高，限制了距离核的可见加速；IVF 中 centroid 距离和倒排链扫描更集中地调用内积核，SIMD 收益更明显。8 ranks 时 SIMD 加速比下降，是因为本地扫描缩短后通信与 merge 占比提高，Amdahl 定律中不可向量化部分变大。

9 图索引组合分析

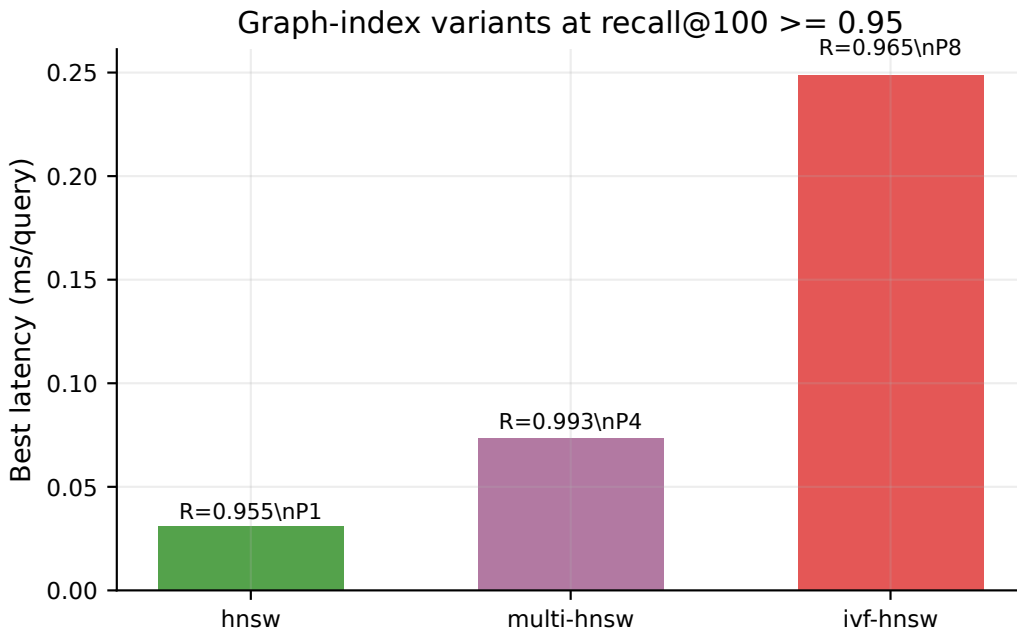


图 8: 图索引路径在 recall@100 $\geq$ 0.95 下的最优延迟对比。

HNSW 的最优点是 0.0312 ms/query, multi-HNSW 为 0.0734 ms/query, IVF-HNSW 为 0.2488 ms/query。multi-HNSW 的 recall 高于单 rank HNSW, 是因为多个分片各自返回 top-100, 合并后候选覆盖面更大; 但查询时每个 rank 都需要执行一次图搜索, 且 root 需要合并更多候选, 延迟没有继续下降。IVF-HNSW 的负担更重: 先要计算 centroid 并选择 nprobe 个簇, 再对多个簇内 HNSW 查询, 等价于在 query 内部串行执行多次小图搜索。当前数据规模下, 小图搜索的常数开销和多簇调度成本高于直接在本地分片上使用一个 HNSW。

图索引的另一个特点是构建时间。HNSW 构建需要逐点插入并维护邻接结构, 1 rank 的构建时间约 6.42 s; 2 ranks 降到约 2.77 s, 4 ranks 降到约 1.31 s。构建可以从 MPI 分片中获益, 但查询阶段是否获益取决于 search、comm、merge 三者占比。若系统以大量离线构建、在线低延迟查询为主, HNSW 是更优选择; 若需要频繁更新或快速重建索引, IVF 的构建成本更低, 工程上更稳健。

10 性能分析与进一步优化

10.1 负载均衡

连续 base 切分使每个 rank 的向量数最多相差 1。Flat 与 HNSW 的 work imbalance 为 1.0; IVF 的 work imbalance 约 1.03, 来自不同局部倒排链长度差异。这个数值说明当前主要问题不是数据量倾斜, 而是阶段比例变化: 当本地搜索加速后, root 端 merge 逐渐固定为每 query 的不可并行部分。

10.2 cache locality

MPI 分片改善了 cache locality。以 Flat-SIMD 为例, 100k base 下 8 ranks 的速度超过理想线性加速, 说明每 rank 扫描的本地数组更小, L2/L3 cache 命中率和堆数据局部性更好。IVF 也从分片中

受益，倒排链变短后候选扫描更快；但 nprobe 过大时会访问许多短链，链切换和 top-k 堆操作的常数成本上升，导致 latency 与 nprobe 接近线性增长。

10.3 通信和 merge

通信量与 $P \cdot nq \cdot k$ 成正比。当前每个候选包含一个 float 距离和一个 32-bit id，1000 query、8 ranks、k=100 时，距离和 id 合计约 6.4 MB，不算大。但 root 端 merge 必须对每个 query 处理 $P \times 100$ 候选，且该阶段没有被 MPI 分散。后续可以考虑两种优化：第一，使用树形 merge/reduce，让中间 rank 先合并部分候选；第二，让每个 rank 返回 top-p 且 $p < 100$ ，以 recall 损失换取更低通信和 merge 成本。这两种方法都会改变 recall-latency 曲线，需要重新以 recall@100 阈值筛选。

10.4 算法选择

在全量数据上，HNSW 是最低延迟方案，IVF 是最低构建成本和较好可解释性的方案。IVF 的最佳点为 nprobe=32，它没有追求 recall=1.0，而是在 recall@100 超过 0.95 后尽量减少访问链数。Flat 只作为精确基线，不适合最终低延迟提交。IVF-HNSW 当前不是最优，但它验证了 IVF 与图索引嵌套结构的可行性：当单簇数据更大、查询批量更大、或者可以并行搜索多个簇时，该结构仍可能有价值。

11 结论

本文完成了 ANN 查询阶段的 MPI 数据分片、局部搜索、候选收集和全局 top-100 合并，并在同一框架下实现了 Flat、IVF、HNSW、multi-HNSW 与 IVF-HNSW。x86 正式实验表明，MPI 对扫描型算法有稳定加速，8 ranks 的 IVF 在 nprobe=32 下达到 0.0573 ms/query、recall@100=0.9654；图索引在低延迟区域更有优势，2 ranks + 4 OpenMP threads 的 HNSW 达到 0.0330 ms/query、recall@100=0.9816。通信方式的差异在单机上小于算法和参数差异；OpenMP 混合并行在本地工作量充足时有效，在 rank 过多时收益下降；SIMD 对距离计算仍有 2-3 $\times$ 的可见贡献，但其上限受 top-k、通信和 merge 约束。

后续若在多节点集群继续扩展，应重点优化全局 merge 和 query batch 流水线。对于 IVF，可尝试树形 top-k reduce、top-p 自适应返回和全局 centroid 同步；对于 HNSW，可探索入口点分配和分片图间剪枝策略。ARM/Kunpeng 侧已保留 quick/full 和 qsub 命令，使用 `-march=native` 后会在 AArch64 上走 NEON 路径。

参考文献

- [1] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE TPAMI*, 2020.
- [2] W. Fu, C. Xiang, Y. Wang, and D. Cai. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *PVLDB*, 2019.
- [3] Message Passing Interface Forum. MPI: A Message-Passing Interface Standard.
- [4] A. André, A.-M. Kermarrec, and N. Le Scouarnec. Cache locality is not enough: high-performance nearest neighbor search with product quantization fast scan. *VLDB*, 2015.

A 生成式 AI 辅助学习记录

问答一：为什么 MPI 版本更适合 base-sharding，而不是 query-sharding？

用户问题：在 ANN 检索中，如果有多个 MPI 进程，为什么要把 base data 切分给不同进程，而不是把 query 分给不同进程？

AI 回答：query-sharding 和 base-sharding 优化的是不同目标。query-sharding 的通信最简单，每个进程处理一部分 query，但前提是每个进程都有完整 base 或完整索引；否则某个 query 只能看到局部数据，返回的不是全局 top-k。如果复制完整索引，内存占用会随进程数线性放大，而且多个进程重复持有同一份 base，不能体现分布式数据并行。base-sharding 则把数据集切成不重叠分片，每个进程只负责自己的向量集合；query 广播给所有进程后，每个进程返回局部 top-k，root 再合并成全局 top-k。这种方法通信更多，但结果语义清晰，能保证 recall 与单机全量检索可比。对于 DEEP100K 这种固定 top-100 评测，base-sharding 的通信量是 $O(PQK)$ ，而计算量从 $O(QNd)$ 降为 $O(QNd/P)$ 。当 N 和 d 较大时，计算下降通常能覆盖候选回收开销；当本地索引非常快时，merge 和通信会成为瓶颈。

问答二：IVF 的 nprobe 为什么会形成 recall-latency trade-off？

用户问题：IVF 中 nprobe 增大时 recall 和 latency 为什么会同时上升？MPI 分片后这个关系有什么变化？

AI 回答：IVF 先把 base 划分到若干倒排簇，查询时只访问与 query 最近的若干簇。nprobe 是被访问的簇数。nprobe 小时，扫描候选少，latency 低，但真实近邻可能落在未访问簇中，recall 降低。nprobe 大时，覆盖更多候选，recall 提高，但需要计算和扫描更多倒排链，latency 上升。MPI 分片后，每个 rank 只拥有本地数据的 IVF 索引，因此本地倒排链长度大约缩短到 $1/P$ ，同一 nprobe 下每个 rank 的扫描量下降；但所有 rank 都返回 top-k，root 的候选池扩大到 $P \times k$ 。这会带来两个结果：一方面，本地搜索变快；另一方面，通信和全局 merge 增大。实验中 nprobe=32 是较好的阈值点，因为它刚好越过 recall@100=0.95，而 nprobe=64/128/256 的额外召回收益不足以抵消延迟增长。

问答三：为什么 MPI_THREAD_MULTIPLE 不一定更快？

用户问题：MPI 支持多线程调用后，为什么 MPI_THREAD_MULTIPLE 没有显著加速？

AI 回答：MPI_THREAD_MULTIPLE 只是允许多个线程同时调用 MPI，并不自动并行化通信。如果程序结构仍然是“工作线程完成本地搜索，主线程统一 gather”，那么 MPI 调用仍然集中在主线程，MULTIPLE 的能力没有被使用。相反，MPI 运行时为了保证线程安全，可能在内部增加锁或临界区，导致少量额外开销。在当前 ANN 实现中，线程并行发生在 query-level local search，通信发生在每轮搜索结束后，因此 MPI_THREAD_FUNNELED 已经足够。MULTIPLE 更适合流水线模型：例如线程 A 搜索 batch b ，线程 B 发送 batch $b-1$ 的局部候选，线程 C 接收或合并更早的 batch。只有当通信和计算真正重叠时，MULTIPLE 才可能体现优势。

问答四：MPI+OpenMP 的进程数和线程数如何取舍？

用户问题：在 MPI+OpenMP 混行中，为什么进程数和线程数不是越多越好？

AI 回答：MPI 进程数增加会缩小每个进程的本地数据分片，从而减少扫描量和索引访问量；OpenMP 线程数增加会让同一进程内多个 query 并发执行。但二者共享 CPU core、cache 和内存

带宽。若 $P \times T$ 接近或超过硬件并发能力，线程调度、cache 争用、NUMA 访问和 MPI 进程上下文切换都会增加。对于 IVF，线程数在本地候选扫描量较大时有效；当 ranks 很多时，每个 rank 的倒排链已经很短，继续增加线程只会放大调度成本。对于 HNSW，单 query 图遍历依赖较强，因此更适合 query-level 并行；但 HNSW 单次查询已经很快，过多 rank 会让 merge 成本占比升高。混合并行的合理策略是先用 MPI 分片把本地工作量降到合适范围，再用少量 OpenMP 线程填满 core，而不是同时把进程数和线程数都拉满。

问答五：IVF-HNSW 为什么可能慢于直接 HNSW？

用户问题： IVF-HNSW 看起来同时用了倒排和图索引，为什么实验中不一定比直接 HNSW 快？

AI 回答： IVF-HNSW 的优势来自两级过滤：先用 IVF 找相关簇，再在簇内用 HNSW 搜索。但它也叠加了两类开销。第一，查询必须先计算到所有 centroid 的距离并选择 nprobe 个簇；第二，每个被选中簇都要启动一次小 HNSW 搜索。小图上的 HNSW 固定开销并不低，多次启动会削弱剪枝收益。如果 nprobe 太小，recall 不足；如果 nprobe 增大到满足 recall 阈值，多个簇内图搜索的总成本可能超过直接在一个本地 HNSW 上搜索。该结构更适合单簇较大、簇间区分明显、或可以并行搜索多个簇的场景。在 DEEP100K 的当前参数下，直接 HNSW 的图导航已经足够有效，IVF-HNSW 的两级结构没有带来更低 latency。